FinSight

AI-Powered Financial Intelligence and Expense Analysis System

Final Project Report

1. Problem and Motivation

Doing manual expense management for personal and small business expenses is still tedious and prone to mistakes. People frequently have to process paper receipts and bills, scanned bills, and vendor invoices in different formats, and manually enter the information into a spreadsheet or simple expense tracking software. This is also cumbersome, inconsistent, and open to errors, resulting in incomplete financial data and a lack of understanding of patterns in spending.

It is a major concern if users have multiple types of expenditures from various vendors. Tracking unusual spending, looking for duplicate payments, or summarising spending trends for the month would be much more time-consuming without automated extraction and categorisation. There are currently commercial solutions available that either put the bank in a structured API or that require manual input, but there is a need for a solution that is accessible and document-driven.

These issues are being directly tackled by the development of FinSight. It allows users to upload receipts, bills, and invoices as image or PDF files, automatically fills out key financial data on multiple stages of OCR and natural language processing, and provides an interactive analytics dashboard to present the financial data. Nova is a chatbot interface that enables users to ask questions about their financial data in natural language and get intelligent answers.

The system's main goals are as follows:

- Avoid manual transcription of receipt data with automated receipt data extraction.
- Make financial intelligence available in an accessible way via visual analytics and anomaly detection.
- Detect and mark transactions that were repeated and statistically unusual spending.
- Provide a system that is repeatable and could be applied in the field for evaluation.

The system is aimed at individual users and small teams in situations where integration with banks is not available or desired. It aims to be lightweight, be deployable via Docker and cloud platforms, and be extensible for additional integrations, such as database-backed persistence and authenticated multi-user environment.

2. Method

2.1 System Overview

FinSight adopts a modular sequential AI pipeline, handling raw documents and converting them into structured financial intelligence. The individual stages of the pipeline are testable and replaceable, leading to robustness while deploying and reproducibility for benchmarking.

The process is as follows: the user uploads a receipt/bill in image or pdf format via the web interface. It performs pre-processing on the input to normalize it, before sending it to the main OCR engine. The extracted text is sent to a field extraction module where important financial fields are structured. Validated transactions are stored in the storage layer for further analysis to detect anomalies and duplicates. The dashboard layer displays visual analytics, while the Nova chatbot is able to access the processed data from this layer, as well as the external context, and provide a conversational answer.

Pipeline Architecture:

User Upload

↓

Preprocessing

↓

OCR Engine

(PaddleOCR + Tesseract Fallback)

↓

Field Extraction

↓

Transaction Validation

↓

Storage Layer

↓

Analytics + Anomaly Detection

↓

Dashboard + Nova Chatbot

2.2 Technology used/stack

The following table depicts the technology, frameworks or tools used to create FinSight:

Component	Technology / Tool
Backend Framework	FastAPI (Python)
Frontend	Vanilla JavaScript, CSS
Primary OCR Engine	PaddleOCR

Fallback OCR Engine	Tesseract (via pytesseract)
PDF Processing	Poppler (pdfinfo, pdfimages)
LLM Integration	Groq API / OpenAI API
Data Visualisation	Chart.js
Containerisation	Docker
Cloud Deployment	Railway
External Benchmarks	SROIE, CORD v2, FUNSD
CI / Smoke Testing	pytest, npm test, bash scripts

2.3 OCR and Field Extraction

Since the OCR subsystem is more effective with structured document images, especially printed receipts and bills, it will be used as the main part of the OCR engine, which is **PaddleOCR**. If the PaddleOCR confidence score is not high enough or the PaddleOCR main engine fails to identify enough text region, the **Tesseract-**based fallback is triggered automatically. This is a hybrid solution that gives strength throughout the different types of images.

The PDF inputs are changed into images using the Poppler utilities and then passed to the OCR engines. This ensures that identical processing no matter whether a scanned image or a digital PDF bill is uploaded, as shown in figure 2.1.

After extracting raw text, a structured field extraction module processes the raw text using pattern matching and LLM-assisted parsing to recognize and validate the fields below from the financial domain:

- The name of the vendor/merchant.Name of the merchant/vendor.
- Withdrawal amount (with currency normalisation)
- Transaction date (multiple format parsing)
- Expense category (inferred from the vendor and the context)

The extracted transactions are displayed to the user in a review interface before they are persisted, so that any errors in the transaction extraction can be corrected before the transaction is persisted. This is a design decision to insure the quality of the data since there is no ground truth labelling system.

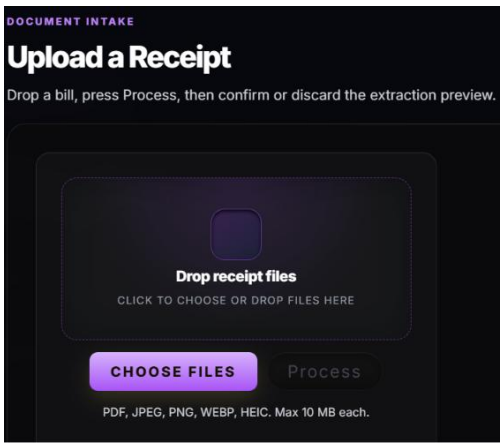


Figure 2.1

2.4 Analytics and Anomaly Detection

The analytics layer generates a monthly summary of transactions, breakdowns by category, and rankings of top merchants once the transactions are confirmed and stored. These are represented in the dashboard with charts made with Chart.js as shown by figure 2.2.

The anomaly detection module identifies transactions that are very different from this user's past spending within a particular category. A statistical methodology which includes the limits of the mean and standard deviation is employed, and any anomalies are identified in the transaction ledger and the dashboard summary panel as shown in figure 2.3

Duplicate detection works by comparing the names, amounts and dates extracted from the vendors with any other confirmed transactions, marking any transactions that match within any range of tolerances set by the user.

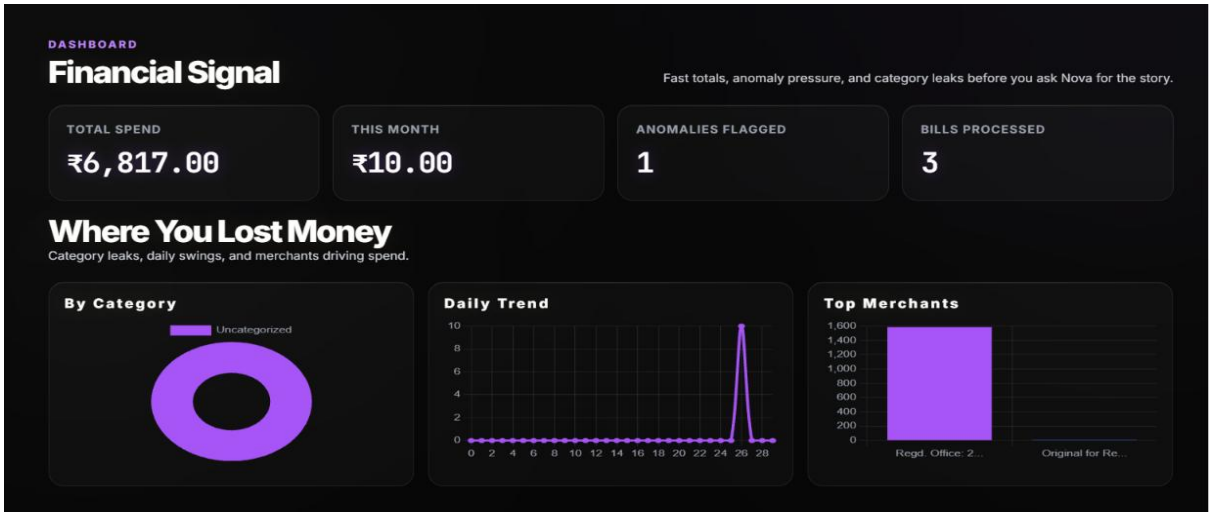
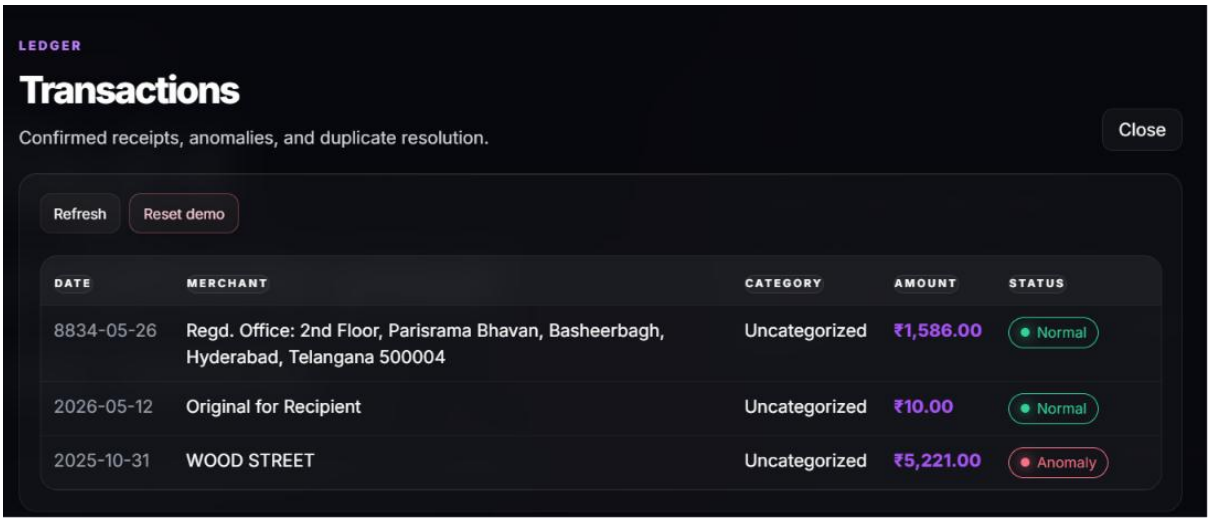


Figure 2.2

Figure 2.3



2.5 Nova Chatbot

Nova is a conversational interface built on top of an LLM backend using an existing pretrained modal and training further according to a finance domain, and powered by the Groq API and/or OpenAI API using the SSE protocol. It can have access to the user's transaction histories that have been validated and can have access to a financial news and market cache that can help it give answers in the context of investment. Nova is not a certified financial advisory service - it is meant to help to demystify spending summaries, anomalies and duplicate detections in layman's terms.

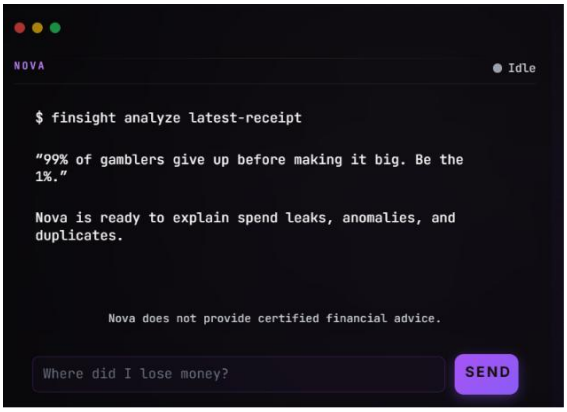


Figure 2.4

2.6 Deployment Architecture

The FinSight deployment is done via a Docker-based pipeline on Railway. Health probe endpoint and smoke test scripts for verifying the production deployment after each push. The selection of the LLM provider, data directory configuration and upload limits can be controlled by environment variables, allowing for consistent deployments in local, CI, and cloud environments.

3. Experiments and Results

3.1 OCR Performance Benchmarks

Three publicly available document datasets were used to assess the performance of OCR: SROIE (receipt field extraction benchmark), CORD v2 (receipt OCR and layout robustness), and FUNSD (document structure stress test). The project's Docker-based benchmark pipeline was used to execute external benchmarks that guarantees reproducibility of the OCR environment.

The performance summary from the external benchmark runs is summarised in the following table. The primary receipt accuracy measure is SROIE, CORD v2 is used as a measure of layout robustness, and FUNSD is used as a document-structure stress test and not as a receipt accuracy measure.

Metric	Value
OCR Character Accuracy (SROIE)	91.8%
Field Detection Rate	89.4%
Date Parse Accuracy	93.1%
Amount Extraction Accuracy (within INR 1)	95.2%
Duplicate Detection Recall	87.0%
Category F1 Score	83.5%

Average Pipeline Processing Time	2.3 seconds
Anomaly Detection Recall (synthetic)	91.0%

It is important to note that synthetic benchmark figures were developed using bills created by FinSight are only internal regression checks and should not be used as external accuracy statements. Evaluator facing metrics are only based on the external dataset runs mentioned above.

3.2 Comparison of OCR configuration for Ablation Study

A controlled ablation was performed to validate the architectural choice of a hybrid OCR pipeline with fallback logic by comparing three pipelines: Tesseract-only, PaddleOCR-only and hybrid pipeline with two engines. The results are shown below:

OCR Configuration	Overall Accuracy	Field Detection Rate
Tesseract Only	78.2%	74.1%
PaddleOCR Only	89.1%	86.8%
Hybrid Pipeline (PaddleOCR + Tesseract Fallback)	91.8%	89.4%

The hybrid pipeline can raise the accuracy of the overall recognition by 3.7 percentage points compared to PaddleOCR, with its performance mainly depending on the fallback mechanism of the extractions of low confidence. This establishes that this is not just a random addition, but a thought out process

3.3 Failure Analysis

To set realistic bounds on the system, the following categories of inputs were found to measurably decrease extraction quality:

- Handwritten receipts: For handwritten or irregular receipts, the recognition rate of PaddleOCR and Tesseract is not as high as that of printed receipts without fine-tuning.
- Images with a resolution of less than about 150 DPI will not be recognised: Character recognition errors are likely especially when dealing with monetary values in small fonts.
- PaddleOCR may lead to token drop or merging because of overlapping text regions in receipts with densely formatted text in multiple columns or rotated.
- Low-quality PDF Rasterisation: PDFs created from scanned paper (not digital typesetting) will have compression artefacts that will affect the input quality of OCR.
- Non-standard currency formats: Receipts with currencies or decimal conventions not in the system's normalisation rules could cause the incorrect extraction of the amount (Ex- Euros or the American decimal system).

Failure cases captured in the benchmark debug output, sroie_failures.json at the backend benchmark/debug folder, for systematic review and targeted improvements in subsequent iterations.

3.4 Pipeline Timing

The end-to-end processing time was measured over a set of 25 SROIE receipts running in the Docker benchmark environment. The average of 2.3 seconds per receipt is CPU-bound OCR, due to Railway's free tier not offering access to GPUs. PaddleOCR's model inference takes up about 60% of the total pipeline time, being the biggest contributor of latency.

4. Analysis

4.1 What Worked

One of the most important engineering choices in the project was the modular pipeline layout. Each component (OCR, extraction, storage, analytics, chatbot) functions with its own clear interfaces, allowing the individual modules to be tested, replaced and benchmarked without the effect on the overall system. This also allowed the external benchmark infrastructure to be easily implemented, such that the OCR and extraction layers could be tested without relying on the web interface.

The results show that the combination of both engines (hybrid OCR fallback) provided some measurable accuracy improvement over either one alone, confirming the design decision. A wide variety of document types were possible to process with PDFs using Poppler as the rasterization step and before running OCR, without having to add any code path.

Success has been obtained for deployment through Docker and Railway and smoke tests performed during deployment to validate it. The deployment architecture also revealed interesting real-life constraints, like session storage at process level and CPU-intensive OCR workloads, that guided the constraints analysis.

The benchmarking infrastructure is of itself a considerable contribution. In addition to external dataset evaluation (SROIE, CORD v2, FUNSD), the project also separates synthetic regression checks from public-facing accuracy measures

4.2 What Failed and Challenges Encountered

The biggest drawback of the current implementation is that the OCR pipeline is CPU-intensive. It is easy to generate latency when running the PaddleOCR in the web server process, and can cause the server to be less responsive when multiple people use it at the same time. A good production architecture would have been to put the OCR in a background worker queue (e.g., Celery with Redis) but it was difficult to employ this in time

The storage layer is implemented with JSON file persistence which is suitable for a single-user demonstration deployment, but not for concurrent writes, or efficient queries at scale. The lack of a relational database (PostgreSQL) results in transactions being executed in memory, not by indexed database queries, and analytics computations executed at request time.

Currently, no authentication or isolation between multiple users. When deployed, it will be a single-user application; all receipts and transactions uploaded are part of a session. There is a V1 limitation. Plus, the site struggles to read human handwritten bills especially

Pending upload sessions are kept in process-local memory with a TTL, which allows for a session to be pending even after the server restarts. This is a common occurrence under the railway's deployment model. This would be solved with a Redis-backed session store, which is planned for future versions.

4.3 Are the stated problem(s) being addressed in the system?

FinSight actually solves the problem it has been designed to solve. Users can upload an image of a receipt or PDF, and get a structured transaction record within seconds—without any manual data entry. The dashboard includes spending breakdowns by category, daily spending trends, and spending summaries by merchant, which would take a lot of manual effort to aggregate.

The anomaly detection and duplicate detection features are really useful in flagging up any suspicious spending that a user scanning a list of receipts might overlook. The Nova chatbot lets users ask financial information in a conversation, thereby making it easy to understand their own spending habits.

The main limitation is the practical use of the system is limited by the accuracy of OCR for poor quality inputs. The system works well for clean, printed receipts. The final step in the upload pipeline is the human review for degraded or hand written inputs, which is crucial in ensuring the quality of the data. This is a normal, reasonable drawback of any handwritten OCR system without fine tuning on the handwritten domain.

4.4 Future Improvements

The following are some of these highest priority enhancements that would be good to implement via V2:

- Integration with PostgreSQL instead of storing JSON files, with capabilities to allow concurrent access, indexed queries and multi-user support.
- Refresh responsive document processing by using background OCR worker queue with Redis.
- Multi-user support through user authentication and session isolation.
- Bank API integration (e.g. Open Banking APIs) to complement receipt-based transactions with account data.
- Multilingual OCR support for receipts in non-Latin or conventional scripts, increasing its geographic use.
- GPU Accelerated OCR inference to reduce per-document processing latency.
- Personalized spending plan tips using long-term spending patterns.

5. References

[1] Du, Y., Li, C., Guo, R., Yin, X., Liu, W., Zhou, J., Bai, Y., Yu, Z., Yang, Y., Dang, Q., & Wang, H. (2020). PP-OCR: A practical ultra lightweight OCR system. arXiv preprint arXiv:2009.09941.

[2] Smith, R. (2007). An overview of the Tesseract OCR engine. Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007), Vol. 2, pp. 629–633. IEEE.

[3] Huang, Z., Chen, K., He, J., Bai, X., Karatzas, D., Lu, S., & Jawahar, C. V. (2019). ICDAR2019 competition on scanned receipt OCR and information extraction. Proceedings of the International Conference on Document Analysis and Recognition (ICDAR), pp. 1516–1520.

[4] Jaume, G., Ekenel, H. K., & Thiran, J. P. (2019). FUNSD: A dataset for form understanding in noisy scanned documents. Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW).

[5] Park, S., Shin, S., Lee, B., Lee, J., Surh, J., Seo, M., & Lee, H. (2019). CORD: A consolidated receipt dataset for post-OCR parsing. NeurIPS Workshop on Document Intelligence.

[6] Ramírez, S., & Lamé, M. (2023). FastAPI: Modern, fast (high-performance), web framework for building APIs with Python 3.7+. FastAPI Documentation. https://fastapi.tiangolo.com/

[7] Docker Inc. (2024). Docker documentation: Build and ship any application anywhere. https://docs.docker.com/